

POROČILO - RSO SEMINAR 2025/2026

Osnovni podatki

- **Naslov projekta:** Predelava aplikacije Kamen–Škarje–Papir v Cloud Native Mikro Storitve
 - **Člana skupine:** Bernard Kučina, Filip Merkan
 - **Povezava do organizacije:** <https://github.com/RSO-skupina-0>
 - **Povezava do aplikacije:** <http://kamen-skarje-papir.click>
-

Kratek opis projekta

Projekt predstavlja predelavo obstoječe monolitne aplikacije "Kamen–Škarje–Papir", objavljene v [GitHub organizaciji](#), v sodobno cloud-native arhitekturo z uporabo mikrostoritev. Aplikacija podpira dve različici igre: klasično **KŠP** in **KŠPOV** ("Kamen–Škarje–Papir–Ogenj–Voda") ter je razdeljena na štiri neodvisne mikrostoritve. Igralec proti računalniku odigra sedem iger KŠP oziroma petnajst iger KŠPOV. Po zaključeni partiji sistem razglasi zmagovalca.

Ogrodje in razvojno okolje

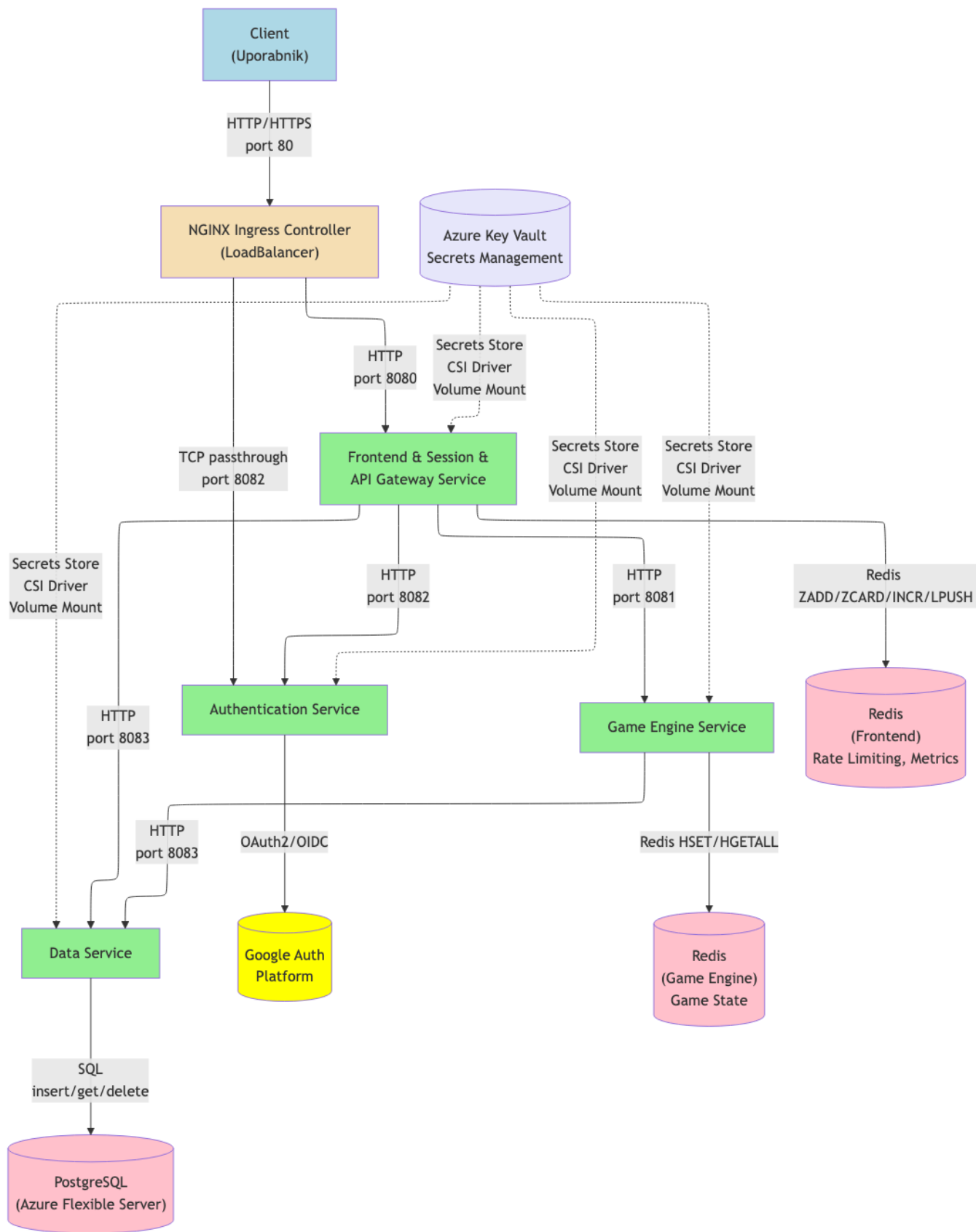
Tehnologije in ogrodja:

- **Backend:** Python 3.11+, spletni strežnik Gunicorn in Bottle spletno ogrodje
- **Baze podatkov:** PostgreSQL (trajno shranjevanje), Redis (seje)
- **Vsebniki:** Docker, Docker Compose
- **Orkestracija:** Kubernetes
- **Komunikacija:** REST API
- **Avtentikacija:** JWT, OAuth 2.0 integracija
- **CI/CD:** GitHub Actions
- **API Gateway:** NGINX

Razvojno okolje

- **IDE:** Visual Studio Code
 - **Upravljanje različic:** Git, GitHub
 - **Testiranje API-jev:** Swagger
 - **Dokumentacija:** Swagger (OpenAPI 3.0), Markdown
-

Shema arhitekture



Seznam funkcionalnosti mikrostoritev

1. Authentication Service

Funkcionalnosti: integracija OAuth2/OIDC (Google) in validacija ID žetonov.

Glavne poti:

- **GET /auth/google/login** – preusmeritev uporabnika na *Google Auth Platform*
- **GET /auth/google/callback** – obdelava povratnega klica iz *Google Auth Platform*
- **GET /auth/redeem** – vrne podatke o prijavljenem uporabniku na podlagi vstopnice

2. Game Engine Service

Funkcionalnosti: implementacija igralne logike za KŠP igr KŠPOV, shranjevanje stanja v trenutne igre na Redis.

Glavne poti:

- **POST /game/ksp/nova** – inicializira novo igro *KŠP*
- **GET /game/ksp/state** – vrne trenutno stanje igre *KŠP*
- **POST /game/ksp/poteza** – izvede potezo in izračuna novo stanje igre *KŠP*
- **POST /game/kspov/nova** – inicializira novo igro *KŠPOV*
- **GET /game/kspov/state** – vrne trenutno stanje igre *KŠPOV*
- **POST /game/kspov/poteza** – izvede potezo in izračuna novo stanje igre *KŠPOV*

4. Data Service

Funkcionalnosti: shranjevanje zaključenih iger (PostgreSQL), pregled zgodovine iger, brisanje zgodovine.

Glavne poti:

- **POST /data/ksp/insert** – zapiše končni rezultat igre *KŠP* v podatkovno bazo
- **GET /data/ksp/get/id** – pridobi ustrezen identifikator igre
- **GET /data/ksp/history** – vrne zgodovino iger uporabnika
- **POST /data/ksp/delete** – izbriše zgodovino iger uporabnika
- **POST /data/kspov/insert** – zapiše končni rezultat igre *KŠPOV* v podatkovno bazo
- **GET /data/kspov/get/id** – pridobi ustrezen identifikator igre
- **GET /data/kspov/history** – vrne zgodovino iger uporabnika
- **POST /data/kspov/delete** – izbriše zgodovino iger uporabnika

5. Frontend, Session, API Gateway Service

Funkcionalnosti: usmerjanje zahtev na mikrostoritve, prikaz uporabniškega vmesnika (HTML/CSS/JS), upravljanje sej (piškotki), omejevanje hitrosti zahtevkov (rate limiting), porazdeljevanje bremena med mikrosotiravmi (load balancing).

Glavne poti:

- **GET /** – prikaz strani za prijavo uporabnika
- **PUT /frontend/login** – prijava uporabnika (*Gost* ali *Uporabnik*) in nastavitve piškotkov
- **GET /auth/finalize** – klic mikrostoritve *Authentication Service* za dokončanje avtentikacije
- **GET /igra** – prikaz začetnega menija za igranje iger *KŠP* in *KŠPOV*
- **POST /ksp** — pridobi podatke o uporabnikovi izbiri (kamen, škarje ali papir)
- **GET /ksp** — iz piškotkov prebere podatke o uporabniku, pridobi stanje igre *KŠP*
- **GET /nova_igra_ksp** — inicializacija nove igre *KŠP*
- **GET /zgodovina_ksp** — pridobi podatke o zgodovini iger
- **DELETE /brisi_ksp** — izbriše zgodovino iger *KŠP* (klic mikrostoritve *Data Service*)
- **POST /kspov** — pridobi podatke o uporabnikovi izbiri (kamen, škarje, papir, ogenj ali voda)
- **GET /kspov** — iz piškotkov prebere podatke o uporabniku, pridobi stanje igre *KŠPOV*
- **GET /nova_igra_kspov** — inicializacija nove igre *KŠPOV*
- **GET /zgodovina_kspov** — pridobi podatke o zgodovini iger *KŠPOV*
- **DELETE /brisi_kspov** — izbriše zgodovino iger *KŠPOV* (klic mikrostoritve *Data Service*)

Primeri uporabe

Osnovni primeri uporabe:

1. Prijava naročnika

- Naročnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Naročnik klikne gumb »**Prijava Google**«.
- Sistem naročnika preusmeri na OAuth ponudnika.
- OAuth ponudnik po uspešni prijavi vrne avtentikacijsko kodo.
- Naročnik je preusmerjen na začetni meni aplikacije.

2. Prijava uporabnika

- Uporabnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Uporabnik vnese svoje uporabniško ime.
- Uporabnik klikne gumb »**Prijava**«.

3. Prijava gosta

- Gost dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
- Gost klikne gumb »Gost«.

4. Nova igra KŠP/KŠPOV

- Uporabnik/gost/naročnik klikne gumb »KŠP« / »KŠPOV«.
- Prikaže se igralni vmesnik za igro KŠP.
- Ob tem se prek klica mikrororitve **Game Engine Service** ustvari nova instanca igre KŠP.

5. Igranje poteze KŠP

- Uporabnik/gost/naročnik izbere možnost (**kamen/škarje/papir**).
- Mikrororitev **Game Engine Service** izračuna rezultat poteze.
- V igralnem vmesniku se prikaže posodobljen rezultat.

6. Igranje poteze KŠPOV

- Uporabnik/gost/naročnik izbere možnost (**kamen/škarje/papir/ogenj/voda**).
- Mikrororitev **Game Engine Service** izračuna rezultat poteze.
- V igralnem vmesniku se prikaže posodobljen rezultat.

7. Pregled zgodovine KŠP/KŠPOV

- Naročnik zahteva pregled zgodovine iger KŠP/KŠPOV.
- Mikrororitev **Data Service** vrne shranjene rezultate iger KŠP/KŠPOV.
- Prikaže se zgodovina iger KŠP.

8. Brisanje zgodovine

- Naročnik zahteva brisanje zgodovine iger.
- Mikrororitev **Data Service** izbriše podatke iz baze **PostgreSQL**.
- Prikaže se prazna zgodovina iger.

Kompleksnejši primer uporabe:

Tok:

1. Naročnik dostopa do aplikacije prek domene <http://kamen-skarje-papir.click>.
2. Naročnik klikne gumb »Prijava z Googlom« in se uspešno avtenticira.
3. Naročnik klikne gumb »KŠPOV«; mikrororitev **Game Engine Service** ustvari novo instanco igre.
4. Naročnik odigra igro, igralni vmesnik pa prikaže končni rezultat.
5. Mikrororitev **Data Service** shrani rezultat v bazo **PostgreSQL**.
6. Naročnik klikne gumb »Zgodovina«.
7. Mikrororitev **Data Service** pridobi podatke o zgodovini iger KŠPOV.
8. Naročnik zahteva brisanje zgodovine.
9. Prikaže se prazna zgodovina iger.

Seznam opravljenih/vključenih osnovnih in dodatnih projektnih zahtev

Postavka	Opis rešitve
Repozitorij	Repozitorij v GitHub organizaciji RSO-skupina-03 vsebuje izvorno kodo, Dockerfile-e, Kubernetes manifeste (Deployment, Service, ConfigMap), CI/CD pipeline (GitHub Actions), Azure deployment skripte in dokumentacijo. Uporabljena je strategija razvejitve z master branch-om.
Mikrostoritve in »cloud-native« aplikacija	Aplikacija je razdeljena na 4 mikrostoritve (Python 3.11+, Bottle, Unicorn): Authentication Service (OAuth2/OIDC), Game Engine Service (igralna logika KŠP/KŠPOV), Data Service (PostgreSQL), Frontend, Session & API Gateway Service (UI, seje, usmerjanje). Vsaka ima Dockerfile in Kubernetes manifeste (Deployment, Service). Nameščene v AKS z cloud-native principi (health checks, readiness probes, resource limits). PostgreSQL (Azure Flexible Server) za trajno shranjevanje z ločenimi tabelami za KŠP in KŠPOV.
Dokumentacija	Dokumentacija v Markdown formatu: README.md (lokalna/oblačna namestitvev, arhitektura), dokumentacija.md (tehnična dokumentacija), porocilo.md (projekt, funkcionalnosti, primeri uporabe, zahteve). Vključuje arhitekturni diagram (arhi.png). Porocilo.md in dokumentacija.md najdete v documents/markdown .
Dokumentacija API	Vse mikrostoritve izpostavljajo Swagger UI na /docs z OpenAPI 3.0 specifikacijo (Bottle-Swagger). Dokumentacija vključuje endpoint-e (GET, POST, PUT, DELETE), JSON sheme zahtev/odgovorov, HTTP status kode (400, 404, 500) in primere uporabe. Swagger UI omogoča interaktivno testiranje API-jev v brskalniku.
Cevovod CI/CD	GitHub Actions pipeline z dvema workflow datotekama: ci-cd.yml (CI: checkout, Python setup, pytest, gradnja Docker slik, push v Docker Hub) in deploy.yml (CD: Azure login OIDC, AKS credentials, posodabljanje SecretProviderClass, nameščanje Kubernetes manifestov, restart deployment-ov, rollout status). Sproži se ob push-u v master ali ročno (workflow_dispatch). Docker slike (filipmerkan/ksp-*) se avtomatsko posodobijo.
Helm charts	Helm se uporablja le za nameščanje NGINX Ingress Controller v Kubernetes okolju.
Namestitev v oblak	Aplikacija nameščena v Microsoft Azure AKS (regija Sweden Central). Kubernetes gruča ksp-aks-cluster-microservices javno dostopna (Standard_B2s VM, managed identity). Javno dostopna prek domene kamen-skarje-papir.click (NGINX Ingress Controller LoadBalancer). Mikrostoritve v AKS (Kubernetes manifesti), Redis instance v gruči, PostgreSQL na Azure Flexible Server, Azure Key Vault za skrivnosti. Infrastruktura konfigurirana z Azure CLI skriptami in CI/CD pipeline-om.
Preverjanje zdravja	Vse mikrostoritve izpostavljajo /health (liveness) in /ready (readiness) endpoint-e. Liveness preverja proces (HTTP 200), readiness preverja odvisnosti (Redis/PostgreSQL, HTTP 200/503). Kubernetes Deployment manifesti vključujejo livenessProbe in readinessProbe (HTTP GET, interval 10s, timeout 5s, failureThreshold 3). Neuspešen readiness probe odstrani pod iz Service endpoint-ov, neuspešen liveness probe restartira pod.

Postavka	Opis rešitve
Zbiranje metrik	Frontend service zbira metrike z middleware funkcijo v Redis (INCR za števce, LPUSH za log zapise, LTRIM za zadnjih 1000 vnosov). Metrike: skupno število zahtevkov (<code>gateway:metrics:total</code>), po endpointih (<code>gateway:metrics:endpoint:{endpoint}</code>), status kode (<code>gateway:metrics:status:{status}</code>), časi odziva (<code>gateway:metrics:response_times</code>), log zapisi z IP, timestamp-i in endpointi (<code>gateway:metrics:logs</code>).
Izolacija in toleranca napak	Odpornost na napake na več ravneh: health checks (liveness/readiness probes) za avtomatsko zamenjavo nezdravih pod-ov; retry logika v Frontend service-u za ponovne klice; circuit breaker za preprečevanje kaskadnih napak; Redis shranjevanje stanja za hitro obnovitev; Kubernetes visoka razpoložljivost z več replikami. Izolacija mikrorstitev zmanjšuje vpliv napak - ločene Redis instance za Frontend in Game Engine preprečujejo kaskadne napake.
Upravljanje s konfiguracijo	Konfiguracija na več nivojih: okoljske spremenljivke v Deployment manifestih (porti, URL-ji); Azure Key Vault za skrivnosti (session secret, OAuth credentials, DB URL) prek Secrets Store CSI Driver (volume mount); ConfigMap za neobčutljive podatke; Kubernetes Secrets. Konfiguracija ločena od implementacije - spremembe ne zahtevajo ponovne gradnje Docker slik. Vse vrednosti v Kubernetes manifestih, spreminjanje brez ponovnega prevajanja.
Grafični vmesnik	Grafični vmesnik (Bottle) sestavljen iz 6 podstrani: prijava, izbira igre, igralna konzola, prikaz zgodovine iger. Komunikacija z mikrorstivami prek REST API-ja.
API Gateway	Frontend service kot API Gateway: usmerjanje zahtevkov na zaledne storitve (Game Engine 8081, Authentication 8082, Data Service 8083) prek HTTP; upravljanje sej s piškotki (secret iz Azure Key Vault); rate limiting z Redis (po IP in endpoint-u); zbiranje metrik in logging. Integriran z NGINX Ingress Controller za zunanji dostop (<code>kamen-skarje-papir.click</code>).
Ingress Controller	NGINX Ingress Controller nameščen v namespace <code>ingress-nginx</code> z Helm chart-om. Konfiguriran z Ingress manifestom za HTTP routing (port 80) do Frontend na <code>/</code> in TCP passthrough (port 8082) za Authentication. Zunanji dostop prek domene <code>kamen-skarje-papir.click</code> (DNS A zapis na LoadBalancer IP). Upravlja load balancing, health checks in TLS/SSL terminacijo.
IAM, OAuth2, OIDC	Authentication Service implementira OAuth 2.0 / OIDC z Google Auth Platform. Tri tipi uporabnikov: Gost, Uporabnik, Naročnik (Google OAuth2/OIDC). Avtentikacija z enkratnimi vstopnicami (one-time tickets, TTL 60s) v Redis za varno posredovanje podatkov med storitvami. OAuth credentials (client ID/secret) v Azure Key Vault, dostop prek Secrets Store CSI Driver. Google ID tokeni se validirajo z Google's public keys.